1 (text) Stacking:

push 1
push 20
pop() - remove top

20 ← top
1 → pop() [1]

The stack would consist of only b/c stacks are an LIFO ADT, which means Last in first out

→ 20 was Last in so ⇒ was "top"

Then pop() effected top (returned and removed)

push (pop()*2)
↓

pop() ⇒ return 1

push (1*2)

push (2) ⇒ stack now consist of 2

push (0) 0 ← top
2

push(pop()/2)
(0/2)
push (0)

0 ← top
2

[front / top
↓
final stack : 0 → 2]

---

2 (text) Queuing

push (2) 2

push (pop()+4) → push (6) → 6

push (5) ⇒ back (5) 6 front

push (pop()/2) push (6/2) ⇒ push (3)

back 3 5 front

pop()
pop()
} empty queue

[final contents : empty stack]

3 (text) Find in deque

queue → enqueue (2)
enqueue (4)
enqueue (5)
dequeue

singly linked list

(2) (4) (5)

(4) (5) → b/c (2) is oldest / infront

left/front right/back

n ... 2 1 0

front Right

A B C D E ⇒ 5 element = n

4 3 2 1 0

n-1-i 0

n-1-0

n-1-1

8-1-0 =	7
7-1	6
7-2	5
7-3	4

---

final answer

```
find (y, a) {
n = a size

for(i=0; i≤n-1-i; i++) {
  left = n-1-i
  right = i

  if(left == right)
   x = removeFront();
   if (x==y)
     return left

  else {
   leftElement = removeFront()
   rightElement = removeBack()

   if (leftElement ==y)
      return left;
   if (rightElement==y)
      return right;

return -1;
```

This algorithm allows for $O(\frac{n}{2})$ with the n being the size of the array because it searches 2 elements per iteration, which halves the amount of iterations it would take simply traversing through list.

7) Time complexity

- isBalanced()

The time complexity would be O(n) where n is the length of string s because the algorithm iterates through the whole string to compare each char and figure out what case they fall into in our algorithm.

The space complexity would be O(n) because we are storing up to n character in our linked list. This occurs in the worst case that all characters are "opening" characters.

- decodeString ()

The time complexity would be O(n+m), where the n is the length of the input, and m is the length of the output length. This because we must inspect all characters to check against cases and must append the string m amount of times, in which worst case could be severual large number (eg. 200[ab] 300[ab] ...).

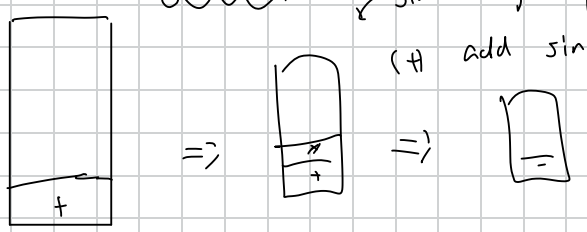
The space complexity would be O(m) where m is the result string since we a creating a String of m length.

- Didn't get to start on infix to postfix code

however learned the common way to use stack is save the operator until lower percedence operator is found.

eg.

a+b*c-d



since higher precen

(+) add since equal

"a" "abc "abc* + d -"

final

This would be O(n) where the n is the length of the string because we iterate through whole string.